

МИНОБРНАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
**«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО»**

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 «Математика и компьютерные науки»

**Отчет о выполнении лабораторных работ по дисциплине
«Методы проектирования баз данных»
по теме «Программирование и настройка СУБД»**

Выполнил студент группы 5130201/40002
Принял доцент к.т.н.

_____ Гасов А.Е.
_____ Попов С.Г.

«___» _____ 2026 г.

Санкт-Петербург
2026

Содержание

Реферат	3
Введение	4
1 Лабораторная работа №1 «Представления»	5
1.1 Задача	5
1.2 Представление	5
1.3 Простой запрос	6
1.4 Сложный запрос	7
2 Лабораторная работа №2 «Триггеры»	10
2.1 Задача	10
2.2 Таблица	11
2.3 Триггер: добавление факта кураторства выставки	11
2.4 Триггер: удаление факта кураторства выставки	13
2.5 Триггер: изменение типа выставки	14
2.6 Триггер: изменение данных сотрудника	15
2.7 Триггер: удаление выставки	16
2.8 Триггер: удаление сотрудника	17
2.9 Схема БД	18
3 Лабораторная работа №3 «Права пользователей»	20
3.1 Задача	20
3.2 Обоснование распределения прав	20
3.3 Назначение прав	20
3.4 Тестовые примеры	21
3.5 Порядок выполнения и результаты	22
Заключение	25
Приложение	26

Реферат

Введение

1 Лабораторная работа №1 «Представления»

1.1 Задача

Реализовать представление на основе таблиц базы данных музея, а также составить простой и сложный запрос, используя данное представление. Простой запрос - это запрос непосредственно к представлению, а в сложном запросе обязано присутствовать соединение представления с другими таблицами базы данных (при помощи JOIN).

1.2 Представление

Условие для формирования представления: Для каждой записи о сотруднике (определяется полями id-сотрудника, именем и фамилией) найти число курируемых выставок и число уникальных типов среди этих выставок.

Текст представления представлен в Листинге 1.

```
1 CREATE VIEW employee_curation_stats AS
2 SELECT
3     employees.employee_id,
4     employees.first_name,
5     employees.last_name,
6     COUNT (exhibition_curators.exhibition_id) AS curated_exhibitions_count,
7     COUNT(DISTINCT exhibitions.exhibition_type_id) AS curated_exhibition_types_count
8
9 FROM
10     employees
11
12     INNER JOIN exhibition_curators
13         ON exhibition_curators.employee_id = employees.employee_id
14     INNER JOIN exhibitions
15         ON exhibitions.exhibition_id = exhibition_curators.exhibition_id
16
17 GROUP BY
18     employees.employee_id,
19     employees.first_name,
20     employees.last_name;
```

Листинг 1: Представление employee_curation_stats

Реляционная формула представления:

$$R = \gamma_{\text{employee_id, first_name, last_name}; \text{COUNT(distinct exhibition_id), COUNT(distinct exhibition_type_id)}} \left(\begin{array}{c} \text{employees} \bowtie_{\text{employee_id}} \\ \text{exhibition_curators} \bowtie_{\text{exhibition_id}} \\ \text{exhibitions} \end{array} \right),$$

где $\bowtie$ — это natural join

1.3 Простой запрос

Текст запроса: Найти все записи о сотрудниках, которые курировали больше 3 выставок и при этом курировали выставки более чем 1 типа.

Запрос обращается к представлению `employee_curation_stats` как к обычной таблице, без соединения с другими таблицами базы данных. Текст запроса представлен в Листинге 2.

```
1 SELECT
2     employee_curation_stats.first_name,
3     employee_curation_stats.last_name,
4     employee_curation_stats.curated_exhibitions_count,
5     employee_curation_stats.curated_exhibition_types_count
6 FROM
7     employee_curation_stats
8 WHERE
9     employee_curation_stats.curated_exhibitions_count > 3
10    AND employee_curation_stats.curated_exhibition_types_count > 1;
```

Листинг 2: Простой запрос к представлению

Реляционная формула запроса:

$$R = \pi_{\text{first_name, last_name, curated_exhibitions_count, curated_exhibition_types_count}} \left(\sigma_{\text{cur_exhibitions_count} > 3 \wedge \text{cur_exhibition_types_count} > 1} (\text{ECS}) \right),$$

где ECS - это `employee_curation_stats`

Результат выполнения запроса (выведены первые 20 строк) представлен на Рис.1.

first_name	last_name	curated_exhibitions_count	curated_exhibition_types_count
Игорь	Петров	4	2
Антон	Андреев	4	4
Дмитрий	Григорьев	4	3
Владимир	Павлов	6	3
Дмитрий	Смирнов	5	5
Степан	Степанов	8	5
Дмитрий	Егоров	4	3
Дмитрий	Лебедев	4	3
Илья	Зайцев	5	3
Иван	Захаров	6	4
Павел	Михайлов	5	4
Дмитрий	Павлов	7	4
Виктор	Волков	6	4
Максим	Яковлев	7	5
Андрей	Семенов	6	5
Николай	Васильев	5	2
Александр	Федоров	7	4
Степан	Алексеев	4	3
Сергей	Никитин	8	5
Денис	Павлов	6	4

Рис. 1: Результат выполнения простого запроса к представлению (выведены первые 20 строк)

План выполнения запроса представлен на Рис.2.

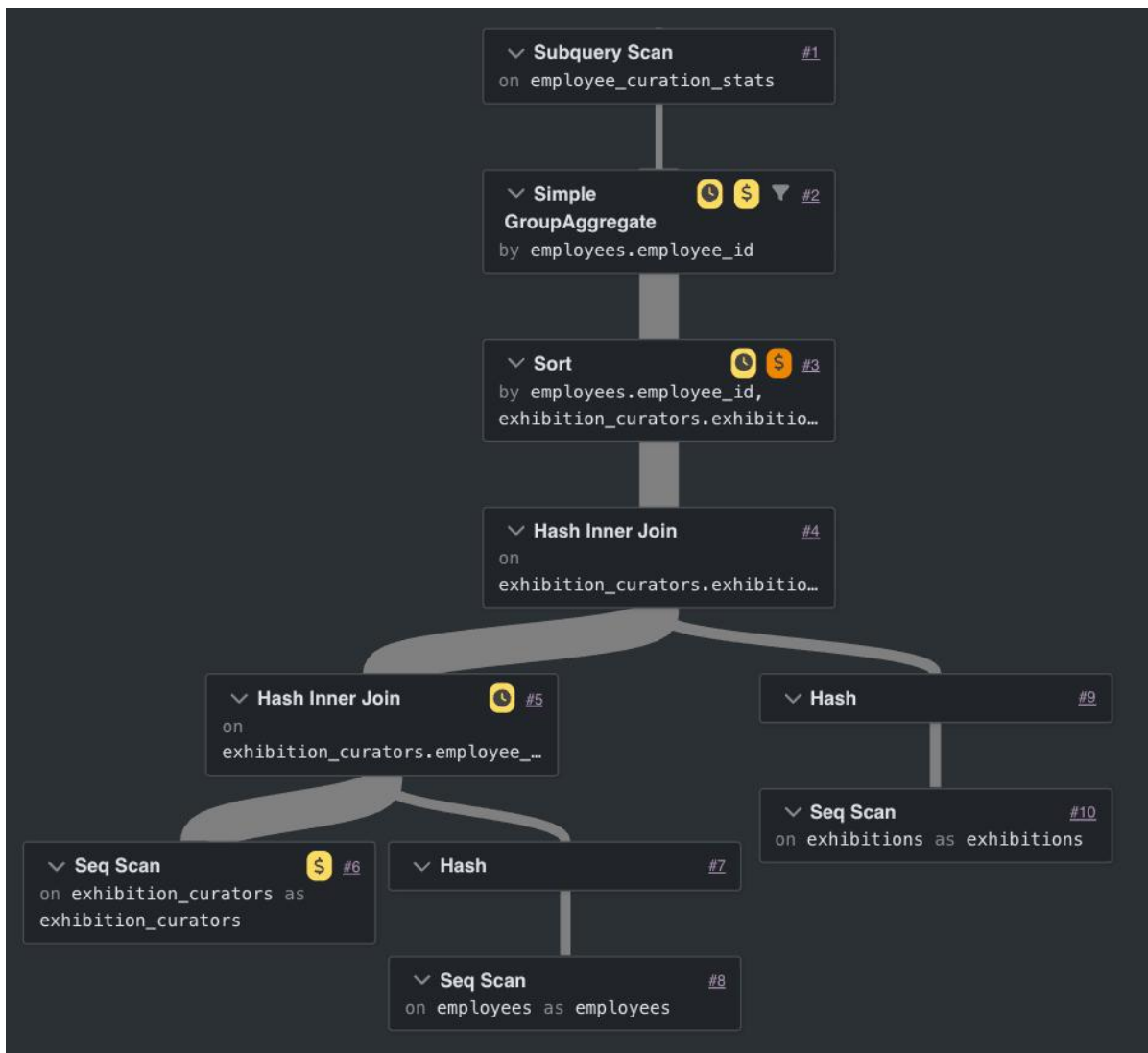


Рис. 2: План выполнения простого запроса к представлению

План разворачивает представление в подзапрос: таблицы `exhibition_curators`, `employees` и `exhibitions` последовательно соединяются через Hash Inner Join, результат сортируется и группируется узлом Simple GroupAggregate. Условие запроса (`curated_exhibitions_count > 3` и `curated_exhibition_types_count > 1`) планировщик применяет прямо внутри этого узла как фильтр после группировки, поэтому верхний узел Subquery Scan лишь передаёт уже отфильтрованные строки.

1.4 Сложный запрос

Текст запроса: Найти записи о технических паспортах, которых составили сотрудники с 2 и более курируемыми выставками.

Запрос соединяет (через JOIN) представление `employee_curation_stats` с таблицей `passports`. Текст запроса представлен в Листинге 3.

```

1 SELECT
2     passports.passport_id,
3     employee_curation_stats.first_name,
4     employee_curation_stats.last_name,
5     employee_curation_stats.curated_exhibitions_count
6 FROM
7     passports
8
9     INNER JOIN employee_curation_stats
10        ON employee_curation_stats.employee_id = passports.employee_id
11
12 WHERE
13     employee_curation_stats.curated_exhibitions_count >= 2;

```

Листинг 3: Сложный запрос с использованием представления

Реляционная формула запроса:

$$R = \pi_{\text{passport_id, first_name, last_name, curated_exhibitions_count}} \left(\sigma_{\text{curated_exhibitions_count} \geq 2} (\text{passports} \bowtie_{\text{employee_id}} \text{ECS}) \right),$$

где $\bowtie$ - это natural join, ECS - это `employee_curation_stats`

Результат выполнения запроса (выведены первые 20 строк) представлен на Рис.3.

passport_id	first_name	last_name	curated_exhibitions_count
1	Степан	Попов	3
2	Денис	Яковлев	6
3	Артем	Морозов	2
4	Игорь	Борисов	5
5	Павел	Михайлов	5
6	Денис	Иванов	4
7	Александр	Николаев	3
8	Иван	Семенов	6
9	Александр	Николаев	3
10	Владимир	Андреев	3
11	Степан	Новиков	5
12	Сергей	Яковлев	5
13	Илья	Степанов	5
14	Дмитрий	Григорьев	4
15	Денис	Семенов	3
16	Сергей	Степанов	2
17	Максим	Зайцев	3
18	Михаил	Семенов	2
19	Александр	Волков	6
21	Игорь	Андреев	2

Рис. 3: Результат выполнения сложного запроса с использованием представления (выведены первые 20 строк)

План выполнения запроса представлен на Рис.4.

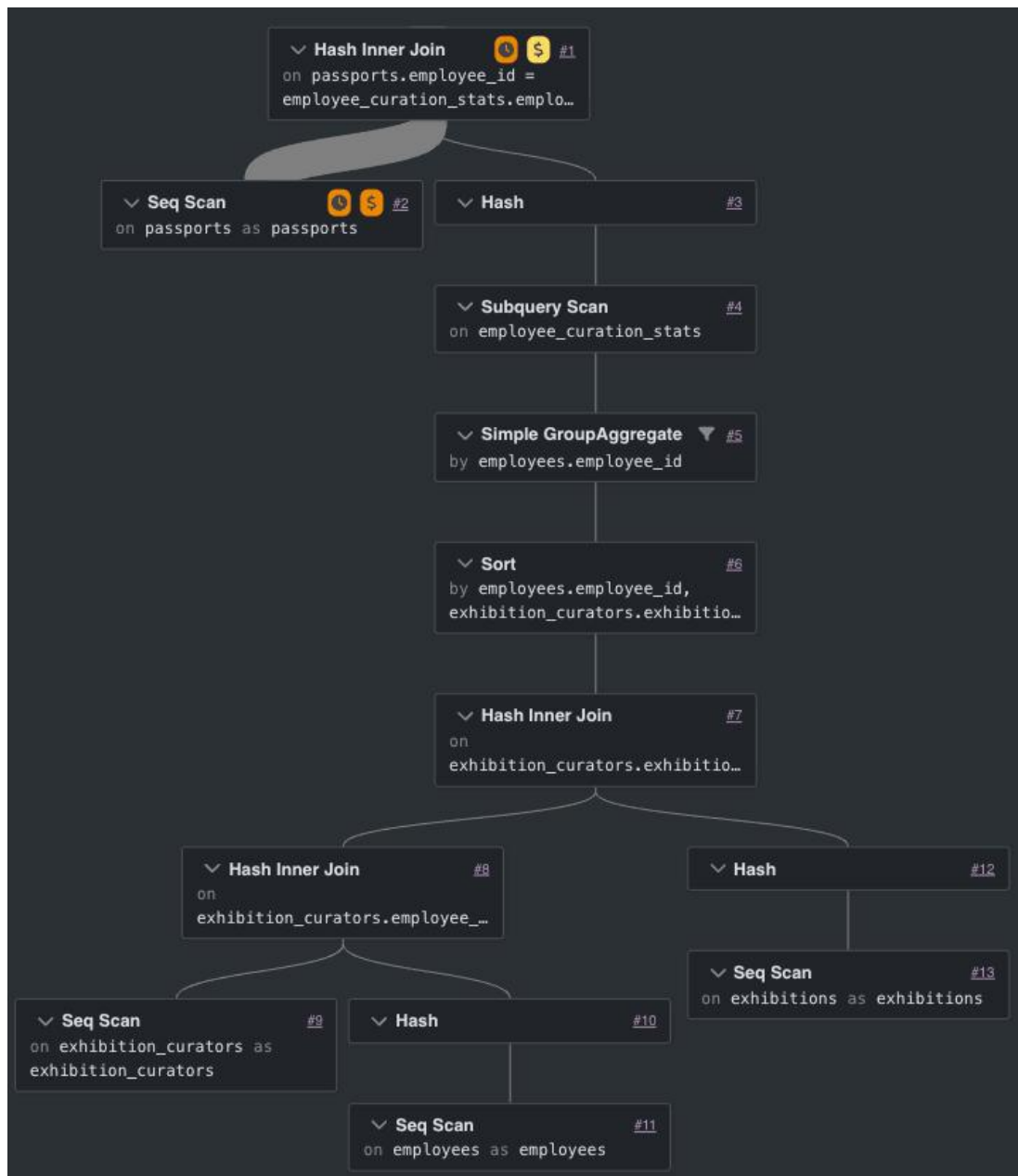


Рис. 4: План выполнения сложного запроса с использованием представления

План аналогичен предыдущему: представление снова разворачивается в тот же набор соединений, сортировку и группировку с фильтром по числу выставок (≥ 2). Сверху добавляется Hash Inner Join с таблицей **passports** — поскольку сторона представления после фильтрации заметно меньше (около 170 сотрудников против 300 тысяч паспортов), хэш-таблица строится именно по ней, а строки **passports** последовательно сканируются и зондируют эту таблицу.

2 Лабораторная работа №2 «Триггеры»

2.1 Задача

Реализовать таблицу, содержание которой повторяет представление `employee_curation_stats`. Обеспечить её актуальность с помощью триггеров: при изменении данных в таблицах `employees`, `exhibitions`, `exhibition_curators` содержимое созданной таблицы должно пересчитываться автоматически.

Должно быть реализовано 6 триггеров, покрывающих следующие изменения:

1. добавление факта кураторства выставки;
2. удаление факта кураторства выставки;
3. изменение типа выставки;
4. изменение данных сотрудника (имя, фамилия);
5. удаление выставки;
6. удаление сотрудника.

Сформированы тестовые примеры, покрывающие все варианты изменения данных исходных таблиц. Результаты выполнения различных действий, влияющих на `employee_curation_stats_table` посредством триггеров, представлены в Таблице 1.

Таблица 1: Примеры работы триггеров

№	Действие
1	Добавление факта кураторства выставки (<code>INSERT</code> в <code>exhibition_curators</code>). Если строки сотрудника в <code>employee_curation_stats_table</code> ещё не было, создаётся новая. Если строка уже была, счётчики пересчитываются.
2	Удаление факта кураторства выставки (<code>DELETE</code> из <code>exhibition_curators</code>). Счётчики сотрудника в <code>employee_curation_stats_table</code> пересчитываются. Если курируемых выставок не осталось, строка удаляется.
3	Изменение типа выставки (<code>UPDATE exhibition_type_id</code> в <code>exhibitions</code>). У всех кураторов этой выставки пересчитывается <code>curated_exhibition_types_count</code> в <code>employee_curation_stats_table</code> .
4	Изменение данных сотрудника (<code>UPDATE first_name, last_name</code> в <code>employees</code>). В <code>employee_curation_stats_table</code> синхронизируются имя и фамилия сотрудника.
5	Удаление выставки (<code>DELETE</code> из <code>exhibitions</code>). У всех её кураторов пересчитываются счётчики в <code>employee_curation_stats_table</code> . Если у кого-то выставок не осталось, его строка удаляется.
6	Удаление сотрудника (<code>DELETE</code> из <code>employees</code>). Соответствующая строка удаляется из <code>employee_curation_stats_table</code> .

2.2 Таблица

Таблица `employee_curation_stats_table` создана на основе представления `employee_curation_stats` и содержит те же поля: идентификатор и имя сотрудника, число курируемых им выставок и число уникальных типов среди них. Текст создания таблицы представлен в Листинге 4.

```
1 CREATE TABLE employee_curation_stats_table AS
2 SELECT
3     employees.employee_id,
4     employees.first_name,
5     employees.last_name,
6     COUNT (exhibition_curators.exhibition_id) AS curated_exhibitions_count,
7     COUNT(DISTINCT exhibitions.exhibition_type_id) AS curated_exhibition_types_count
8
9 FROM
10     employees
11
12     INNER JOIN exhibition_curators
13         ON exhibition_curators.employee_id = employees.employee_id
14     INNER JOIN exhibitions
15         ON exhibitions.exhibition_id = exhibition_curators.exhibition_id
16
17 GROUP BY
18     employees.employee_id,
19     employees.first_name,
20     employees.last_name;
```

Листинг 4: Таблица `employee_curation_stats_table`

2.3 Триггер: добавление факта кураторства выставки

Срабатывает на `AFTER INSERT` по таблице `exhibition_curators`: сотруднику назначена ещё одна выставка для кураторства, его строка в `employee_curation_stats_table` пересчитывается заново. Текст триггера представлен в Листинге 5.

```
1 -- добавление факта кураторства выставки
2 CREATE OR REPLACE FUNCTION trg_curator_added()
3 RETURNS TRIGGER AS $$
4 BEGIN
5     -- обновляем строку, если она уже есть
6     UPDATE employee_curation_stats_table
7     SET curated_exhibitions_count = affected_with_counts.exhibitions_count,
8         curated_exhibition_types_count = affected_with_counts.types_count
9     FROM (
10         SELECT
11             exhibition_curators.employee_id,
12             COUNT(exhibition_curators.exhibition_id) AS exhibitions_count,
13             COUNT(DISTINCT exhibitions.exhibition_type_id) AS types_count
14         FROM exhibition_curators
15             INNER JOIN exhibitions ON exhibitions.exhibition_id = exhibition_curators
16             .exhibition_id
```

```

16      WHERE exhibition_curators.employee_id = NEW.employee_id
17      GROUP BY exhibition_curators.employee_id
18  ) AS affected_with_counts
19  WHERE employee_curation_stats_table.employee_id = affected_with_counts.
    employee_id;
20
21  -- вставляем строку, если её ещё не было за( это отвечает проверка с count = 0)
22  INSERT INTO employee_curation_stats_table
23      (employee_id, first_name, last_name, curated_exhibitions_count,
    curated_exhibition_types_count)
24  SELECT
25      employees.employee_id, employees.first_name, employees.last_name,
26      COUNT(exhibition_curators.exhibition_id),
27      COUNT(DISTINCT exhibitions.exhibition_type_id)
28  FROM employees
29      INNER JOIN exhibition_curators ON exhibition_curators.employee_id = employees
    .employee_id
30      INNER JOIN exhibitions ON exhibitions.exhibition_id = exhibition_curators.
    exhibition_id
31  WHERE employees.employee_id = NEW.employee_id
32      AND (SELECT COUNT(*) FROM employee_curation_stats_table WHERE employee_id = NEW
    .employee_id) = 0
33  GROUP BY employees.employee_id, employees.first_name, employees.last_name;
34
35  RETURN NEW;
36 END;
37 $$ LANGUAGE plpgsql;
38
39 CREATE OR REPLACE TRIGGER curator_added
40     AFTER INSERT ON exhibition_curators
41     FOR EACH ROW
42     EXECUTE FUNCTION trg_curator_added();

```

Листинг 5: Триггер добавления факта кураторства

Проверка триггера (состояние `employee_curation_stats_table` до и после) представлена на Рис.5.

```

=== ТЕСТ 1: добавление факта кураторства (INSERT exhibition_curators) ===
--- ДО: сотрудник ещё ничего не курирует, строки по нему нет ---
 employee_id | first_name | last_name | curated_exhibitions_count | curated_exhibition_types_count
-----+-----+-----+-----+-----
(0 rows)

INSERT 0 1
--- ПОСЛЕ: появилась строка, curated_exhibitions_count = 1, curated_exhibition_types_count = 1 ---
 employee_id | first_name | last_name | curated_exhibitions_count | curated_exhibition_types_count
-----+-----+-----+-----+-----
      199   | ТестИмя   | ТестФамилия |          1 |          1
(1 row)

```

Рис. 5: Проверка триггера добавления факта кураторства

2.4 Триггер: удаление факта кураторства выставки

Срабатывает на AFTER DELETE по таблице `exhibition_curators`: у сотрудника сняли кураторство над выставкой, его строка пересчитывается. Если курируемых выставок не осталось, строка сотрудника пропадает из таблицы. Текст триггера представлен в Листинге 6.

```
1  -- удаление факта кураторства выставки
2  CREATE OR REPLACE FUNCTION trg_curator_removed()
3  RETURNS TRIGGER AS $$
4  BEGIN
5      -- обновляем счётчики, если у сотрудника остались другие выставки
6      UPDATE employee_curation_stats_table
7      SET curated_exhibitions_count = affected_with_counts.exhibitions_count,
8          curated_exhibition_types_count = affected_with_counts.types_count
9      FROM (
10         SELECT
11             exhibition_curators.employee_id,
12             COUNT(exhibition_curators.exhibition_id) AS exhibitions_count,
13             COUNT(DISTINCT exhibitions.exhibition_type_id) AS types_count
14         FROM exhibition_curators
15             INNER JOIN exhibitions ON exhibitions.exhibition_id = exhibition_curators
16             .exhibition_id
17         WHERE exhibition_curators.employee_id = OLD.employee_id
18         GROUP BY exhibition_curators.employee_id
19     ) AS affected_with_counts
20     WHERE employee_curation_stats_table.employee_id = affected_with_counts.
21         employee_id;
22
23     -- курировать больше нечего - убираем строку
24     DELETE FROM employee_curation_stats_table
25     WHERE employee_id = OLD.employee_id AND (SELECT COUNT(*) FROM exhibition_curators
26         WHERE employee_id = OLD.employee_id) = 0;
27
28     RETURN OLD;
29 END;
30 $$ LANGUAGE plpgsql;
31
32 CREATE OR REPLACE TRIGGER curator_removed
33 AFTER DELETE ON exhibition_curators
34 FOR EACH ROW
35 EXECUTE FUNCTION trg_curator_removed();
```

Листинг 6: Триггер удаления факта кураторства

Проверка триггера (состояние `employee_curation_stats_table` до и после) представлена на Рис.6.

```

=== ТЕСТ 2: удаление факта кураторства (DELETE exhibition_curators) ===
--- ДО: у сотрудника 2 курируемые выставки ---
employee_id | first_name | last_name | curated_exhibitions_count | curated_exhibition_types_count
-----
199 | ТестИмя | ТестФамилия | 2 | 1
(1 row)

DELETE 1
--- ПОСЛЕ: curated_exhibitions_count вернулся к 1 (осталась только В) ---
employee_id | first_name | last_name | curated_exhibitions_count | curated_exhibition_types_count
-----
199 | ТестИмя | ТестФамилия | 1 | 1
(1 row)

```

Рис. 6: Проверка триггера удаления факта кураторства

2.5 Триггер: изменение типа выставки

Срабатывает на AFTER UPDATE OF exhibition_type_id по таблице exhibitions: при смене типа выставки меняется число уникальных типов среди курируемых выставок у всех её кураторов, поэтому строки всех этих сотрудников пересчитываются одним запросом UPDATE. Текст триггера представлен в Листинге 7.

```

1  -- изменение типа выставки -> меняется число уникальных типов
2  -- среди курируемых выставок у всех её кураторов, нужно пересчитать
3  CREATE OR REPLACE FUNCTION trg_exhibition_type_changed()
4  RETURNS TRIGGER AS $$
5  BEGIN
6      -- одним UPDATE пересчитываем сразу всех кураторов этой выставки, без цикла;
7      -- своя выставка у них никуда не делась, поэтому агрегат точно не пуст
8      UPDATE employee_curation_stats_table
9      SET curated_exhibitions_count = affected_with_counts.exhibitions_count,
10         curated_exhibition_types_count = affected_with_counts.types_count
11      FROM (
12          SELECT
13              exhibition_curators.employee_id,
14              COUNT(exhibition_curators.exhibition_id) AS exhibitions_count,
15              COUNT(DISTINCT exhibitions.exhibition_type_id) AS types_count
16          FROM exhibition_curators
17              INNER JOIN exhibitions ON exhibitions.exhibition_id = exhibition_curators
18                  .exhibition_id
19              WHERE exhibition_curators.employee_id IN (
20                  SELECT employee_id FROM exhibition_curators WHERE exhibition_id = NEW.
21                  exhibition_id
22              )
23          GROUP BY exhibition_curators.employee_id
24          ) AS affected_with_counts
25      WHERE employee_curation_stats_table.employee_id = affected_with_counts.
26      employee_id;
27      RETURN NEW;
28  END;
29  $$ LANGUAGE plpgsql;

```

```

28
29 CREATE OR REPLACE TRIGGER exhibition_type_changed
30     AFTER UPDATE OF exhibition_type_id ON exhibitions
31     FOR EACH ROW
32     EXECUTE FUNCTION trg_exhibition_type_changed();

```

Листинг 7: Триггер изменения типа выставки

Проверка триггера (состояние `employee_curation_stats_table` до и после) представлена на Рис.7.

```

=== ТЕСТ 3: изменение типа выставки (UPDATE exhibitions.exhibition_type_id) ===
--- ДО: А и В одного типа, curated_exhibition_types_count = 1 ---
employee_id | first_name | last_name | curated_exhibitions_count | curated_exhibition_types_count
-----
199 | ТестИмя | ТестФамилия | 2 | 1
(1 row)

UPDATE 1
--- ПОСЛЕ: у А теперь другой тип, curated_exhibition_types_count = 2 ---
employee_id | first_name | last_name | curated_exhibitions_count | curated_exhibition_types_count
-----
199 | ТестИмя | ТестФамилия | 2 | 2
(1 row)

```

Рис. 7: Проверка триггера изменения типа выставки

2.6 Триггер: изменение данных сотрудника

Срабатывает на `AFTER UPDATE OF first_name, last_name` по таблице `employees`: при смене имени или фамилии сотрудника эти же поля синхронизируются в его строке `employee_curation_stats_table`. Счётчики не пересчитываются - смена имени на них не влияет. Текст триггера представлен в Листинге 8.

```

1  -- изменение сотрудника имяфамилия(/) -> синхронизируем в таблице
2  -- счётчики не пересчитываем - смена имени на них не влияет
3  CREATE OR REPLACE FUNCTION trg_employee_renamed()
4  RETURNS TRIGGER AS $$
5  BEGIN
6      UPDATE employee_curation_stats_table
7      SET first_name = NEW.first_name,
8          last_name = NEW.last_name
9      WHERE employee_id = NEW.employee_id;
10     RETURN NEW;
11 END;
12 $$ LANGUAGE plpgsql;
13
14 CREATE OR REPLACE TRIGGER employee_renamed
15     AFTER UPDATE OF first_name, last_name ON employees
16     FOR EACH ROW
17     EXECUTE FUNCTION trg_employee_renamed();

```

Листинг 8: Триггер изменения данных сотрудника

Проверка триггера (состояние `employee_curation_stats_table` до и после) представлена на Рис.8.

```

=== ТЕСТ 4: изменение сотрудника (UPDATE employees, смена имени) ===
--- ДО: сотрудника зовут ТестИмя ТестФамилия ---
employee_id | first_name | last_name | curated_exhibitions_count | curated_exhibition_types_count
-----
199 | ТестИмя | ТестФамилия | 2 | 2
(1 row)

UPDATE 1
--- ПОСЛЕ: first_name синхронизировался, счётчики не изменились ---
employee_id | first_name | last_name | curated_exhibitions_count | curated_exhibition_types_count
-----
199 | ТестИмяПереименован | ТестФамилия | 2 | 2
(1 row)

```

Рис. 8: Проверка триггера изменения данных сотрудника

2.7 Триггер: удаление выставки

Срабатывает на `BEFORE DELETE` по таблице `exhibitions`: сначала запоминаются `id` всех кураторов удаляемой выставки, затем убираются их связи в `exhibition_curators` (несмотря на `ON DELETE CASCADE`, триггер сам чистит связи заранее, т.к. он `BEFORE DELETE`, а каскад выполнится только после него), после чего строки задетых сотрудников пересчитываются одним пакетным запросом. Текст триггера представлен в Листинге 9.

```

1 -- удаление выставки -> у всех ее кураторов пропадает эта выставка из счета
2 -- поэтому пересчитываем их строки или( удаляем строку, если это была последняя
  выставка)
3 CREATE OR REPLACE FUNCTION trg_exhibition_deleted()
4 RETURNS TRIGGER AS $$
5 DECLARE
6     affected_employees INT[];
7 BEGIN
8     -- запоминаем ID всех кураторов этой выставки, пока связи ещё не удалены
9     SELECT ARRAY_AGG(DISTINCT employee_id)
10    INTO affected_employees
11    FROM exhibition_curators
12    WHERE exhibition_id = OLD.exhibition_id;
13
14     -- несмотря на ON DELETE CASCADE, чистим сами: триггер BEFORE DELETE, а каскад
15     -- выполнится только после него - без этой строки пересчёт ниже увидит
    устаревшие связи
16     DELETE FROM exhibition_curators WHERE exhibition_id = OLD.exhibition_id;
17
18     -- одним UPDATE пересчитываем всех задетых сотрудников, у которых остались
    другие выставки
19     UPDATE employee_curation_stats_table
20     SET curated_exhibitions_count = affected_with_counts.exhibitions_count,
21         curated_exhibition_types_count = affected_with_counts.types_count
22     -- подзапросом находим задетых сотрудников
23     FROM (

```

```

24      SELECT
25          exhibition_curators.employee_id,
26          COUNT(exhibition_curators.exhibition_id) AS exhibitions_count,
27          COUNT(DISTINCT exhibitions.exhibition_type_id) AS types_count
28      FROM exhibition_curators
29          INNER JOIN exhibitions ON exhibitions.exhibition_id = exhibition_curators
        .exhibition_id
30      WHERE exhibition_curators.employee_id = ANY(affected_employees)
31      GROUP BY exhibition_curators.employee_id
32  ) AS affected_with_counts
33  WHERE employee_curation_stats_table.employee_id = affected_with_counts.
        employee_id;
34
35  -- DELETE убираем строки тех, у кого выставок больше не осталось
36  DELETE FROM employee_curation_stats_table
37  WHERE employee_id = ANY(affected_employees) AND employee_id NOT IN (SELECT
        employee_id FROM exhibition_curators);
38
39  RETURN OLD;
40 END;
41 $$ LANGUAGE plpgsql;
42
43 CREATE OR REPLACE TRIGGER exhibition_deleted
44     BEFORE DELETE ON exhibitions
45     FOR EACH ROW
46     EXECUTE FUNCTION trg_exhibition_deleted();

```

Листинг 9: Триггер удаления выставки

Проверка триггера (состояние `employee_curation_stats_table` до и после) представлена на Рис.9.

```

=== ТЕСТ 5: удаление выставки (DELETE exhibitions) ===
--- ДО: у сотрудника 2 курируемые выставки (А и В) ---

```

employee_id	first_name	last_name	curated_exhibitions_count	curated_exhibition_types_count
199	ТестИмяПереименован	ТестФамилия	2	2

```

(1 row)

DELETE 1
--- ПОСЛЕ: curated_exhibitions_count = 1 (осталась только В) ---

```

employee_id	first_name	last_name	curated_exhibitions_count	curated_exhibition_types_count
199	ТестИмяПереименован	ТестФамилия	1	1

```

(1 row)

```

Рис. 9: Проверка триггера удаления выставки

2.8 Триггер: удаление сотрудника

Срабатывает на `AFTER DELETE` по таблице `employees`: строка удалённого сотрудника убирается из `employee_curation_stats_table` (связи в `exhibition_curators` к этому моменту уже удалены каскадом). Текст триггера представлен в Листинге 10.

```

1  -- удаление сотрудника -> убираем его строку из таблицы
2  CREATE OR REPLACE FUNCTION trg_employee_deleted()
3  RETURNS TRIGGER AS $$
4  BEGIN
5      DELETE FROM employee_curation_stats_table WHERE employee_id = OLD.employee_id;
6      RETURN OLD;
7  END;
8  $$ LANGUAGE plpgsql;
9
10 CREATE OR REPLACE TRIGGER employee_deleted
11     AFTER DELETE ON employees
12     FOR EACH ROW
13     EXECUTE FUNCTION trg_employee_deleted();

```

Листинг 10: Триггер удаления сотрудника

Проверка триггера (состояние `employee_curation_stats_table` до и после) представлена на Рис.10.

```

=== ТЕСТ 6: удаление сотрудника (DELETE employees) ===
--- ДО: у сотрудника есть курируемая выставка, строка в таблице присутствует ---
employee_id | first_name | last_name | curated_exhibitions_count | curated_exhibition_types_count
-----
209 | ТестИмяПереименован | ТестФамилия | 1 | 1
(1 row)

DELETE 1
--- ПОСЛЕ: строки по сотруднику в таблице больше нет ---
should_be_zero_table
-----
0
(1 row)

```

Рис. 10: Проверка триггера удаления сотрудника

2.9 Схема БД

Схема базы данных на английском языке с добавленной таблицей `employee_curation_stats_table` и триггерами представлена на Рис. 11.

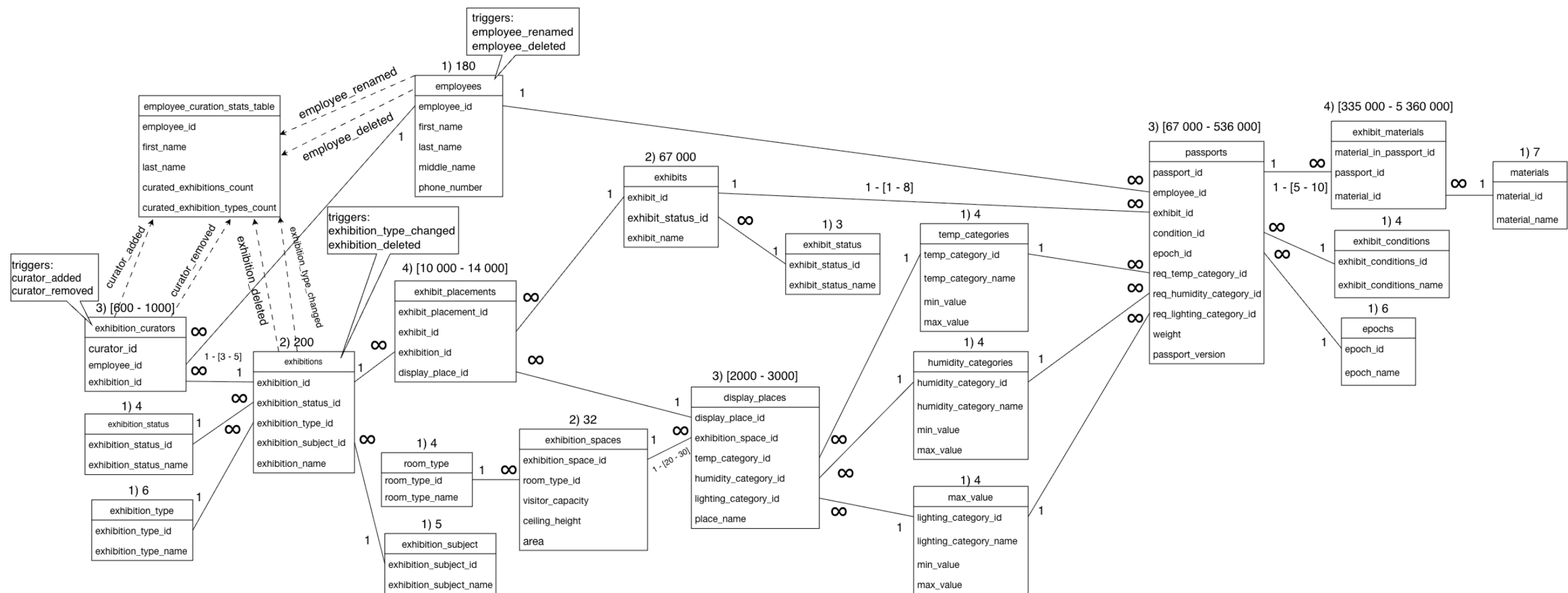


Рис. 11: Схема базы данных на английском языке с учетом триггеров и добавленной таблицы employee_curation_stats_table

3 Лабораторная работа №3 «Права пользователей»

3.1 Задача

Создать двух пользователей базы данных, обладающих различными правами доступа:

1. Пользователь 1 (`guinea_pig`) может только просматривать `employee_curation_stats_table`. Ему доступна только операция `SELECT` из этой таблицы, ко всем остальным таблицам доступа нет.
2. Пользователь 2 (`ex_curator`) имеет полный доступ (`SELECT`, `INSERT`, `UPDATE`, `DELETE`) только к тем таблицам, изменения которых влияют на `employee_curation_stats_table`. К самой `employee_curation_stats_table` у него есть только право просмотра (`SELECT`), ко всем остальным таблицам доступа нет.

3.2 Обоснование распределения прав

Пользователь 1 (`guinea_pig`). Данный пользователь создан для просмотра сводной информации о кураторах выставок (`employee_curation_stats_table`), он не должен видеть исходные таблицы (например, `employees` с телефонами сотрудников) и тем более менять их содержимое. Поэтому данному пользователю выдано только право `SELECT` на одну таблицу.

Пользователь 2 (`ex_curator`). Данный пользователь создан для работы с данными, связанными с кураторами выставок: сотрудниками, выставками и фактами кураторства. Так как в таблице `employee_curation_stats_table` агрегируется информация по кураторству, пользователю было выдано право просмотра данной таблицы. Для вставки новых записей в таблицы `employees`, `exhibitions` и `exhibition_curators` ему также выданы права на последовательности (`SERIAL`) их первичных ключей. Права на запись в `employee_curation_stats_table` у него нет, так как если бы он мог писать в неё напрямую, то значения перестали бы соответствовать реальным данным. В реализованной БД актуальность данных в таблице `employee_curation_stats_table` поддерживается триггерами.

3.3 Назначение прав

Текст SQL-скрипта создания пользователей и назначения прав с комментариями представлен в Листинге 11. Так как таблицы создавались без указания схем, все они находятся в схеме `public`, поэтому обоим пользователям выдаётся право `USAGE` на неё.

```

1 CREATE USER guinea_pig PASSWORD '123';
2 CREATE USER ex_curator PASSWORD '123';
3
4 -- вход в БД и видимость схемы нужны обоим
5 GRANT CONNECT ON DATABASE museum_db TO guinea_pig, ex_curator;
6 -- тк.. при создании таблиц, не были указаны схемы, то все таблицы сейчас лежат в
   public
7 GRANT USAGE ON SCHEMA public TO guinea_pig, ex_curator;
8
9
10 -- для морской свинки только SELECT из новой таблицы
11 GRANT SELECT ON employee_curation_stats_table TO guinea_pig;
12
13 -- ex_curator
14 GRANT SELECT ON employee_curation_stats_table TO ex_curator;
15 GRANT SELECT, INSERT, UPDATE, DELETE ON employees, exhibitions, exhibition_curators
   TO ex_curator;
16
17 -- для INSERT в таблицы с SERIAL нужны права на их последовательности, без них меня
   послало
18 GRANT USAGE, SELECT, UPDATE ON SEQUENCE
19     employees_employee_id_seq,
20     exhibitions_exhibition_id_seq,
21     exhibition_curators_curator_id_seq

```

Листинг 11: Создание пользователей и назначение прав

3.4 Тестовые примеры

Для проверки прав было сформировано 10 запросов, с операциями SELECT, INSERT, UPDATE, DELETE для таблиц трех видов: таблицы-агрегата, исходных таблиц и посторонней таблицы (passports). Текст запросов представлен в Листинге 12. Условия в запросах, изменяющих данные, ссылаются на несуществующие или на тестовые записи, поэтому реальные данные не затрагиваются.

```

1 -- 1. SELECT из новой таблицы оба(: таблица)
2 SELECT * FROM employee_curation_stats_table LIMIT 3;
3
4 -- 2. INSERT в новую таблицу, тестовый id 999999 оба(: permission denied)
5 INSERT INTO employee_curation_stats_table
6     (employee_id, first_name, last_name, curated_exhibitions_count,
7      curated_exhibition_types_count)
8 VALUES (999999, 'Гвинея', 'Пиг', 1, 1);
9
10 -- 3. UPDATE новой таблицы, условие на несуществующий id оба(: permission denied)
11 UPDATE employee_curation_stats_table SET curated_exhibitions_count = 100 WHERE
12     employee_id = 999999;
13
14 -- 4. DELETE из новой таблицы, условие на несуществующий id оба(: permission denied)
15 DELETE FROM employee_curation_stats_table WHERE employee_id = 999999;

```

```

14
15 -- 5. SELECT из исходной таблицы employees (guinea_pig: permission denied, ex_curator
    : таблица)
16 SELECT * FROM employees LIMIT 3;
17
18 -- 6. INSERT тестового сотрудника ТестИмя ТестФамилия (guinea_pig: permission denied,
    ex_curator: INSERT 0 1)
19 INSERT INTO employees (first_name, last_name) VALUES ('ТестИмя', 'ТестФамилия');
20
21 -- 7. UPDATE имени тестового сотрудника на ТестИмя2
22 -- (guinea_pig: permission denied, ex_curator: UPDATE 1)
23 UPDATE employees SET first_name = 'ТестИмя2' WHERE last_name = 'ТестФамилия';
24
25 -- 8. DELETE тестового сотрудника
26 -- (guinea_pig: permission denied, ex_curator: DELETE 1)
27 DELETE FROM employees WHERE last_name = 'ТестФамилия';
28
29 -- 9. SELECT из постороннего объекта passports оба(: permission denied)
30 SELECT * FROM passports LIMIT 3;
31
32 -- 10. DELETE из постороннего объекта passports, условие на несуществующий id оба(:
    permission denied)
33 DELETE FROM passports WHERE passport_id = -1;

```

Листинг 12: Тестовые запросы для проверки прав пользователей

3.5 Порядок выполнения и результаты

Результаты выполнения тестовых запросов представлены в Таблице 2.

Таблица 2: Результаты выполнения тестовых запросов для пользователей guinea_pig и ex_curator

№	guinea_pig	ex_curator
1	SELECT * FROM employee_curation_stats_table LIMIT 3;	
	Первые 3 строки таблицы	Первые 3 строки таблицы
2	INSERT INTO employee_curation_stats_table (employee_id, first_name, last_name, curated_exhibitions_count, curated_exhibition_types_count) VALUES (999999, 'Гвинея', 'Пиг', 1, 1);	
	permission denied for table employee_curation_stats_table	permission denied for table employee_curation_stats_table
3	UPDATE employee_curation_stats_table SET curated_exhibitions_count = 100 WHERE employee_id = 999999;	
	permission denied for table employee_curation_stats_table	permission denied for table employee_curation_stats_table
4	DELETE FROM employee_curation_stats_table WHERE employee_id = 999999;	
	permission denied for table employee_curation_stats_table	permission denied for table employee_curation_stats_table
5	SELECT * FROM employees LIMIT 3;	
	permission denied for table employees	Первые 3 строки таблицы
6	INSERT INTO employees (first_name, last_name) VALUES ('ТестИмя', 'ТестФамилия');	
	permission denied for table employees	INSERT 0 1
7	UPDATE employees SET first_name = 'ТестИмя2' WHERE last_name = 'ТестФамилия';	
	permission denied for table employees	UPDATE 1
8	DELETE FROM employees WHERE last_name = 'ТестФамилия';	
	permission denied for table employees	DELETE 1
9	SELECT * FROM passports LIMIT 3;	
	permission denied for table passports	permission denied for table passports
10	DELETE FROM passports WHERE passport_id = -1;	
	permission denied for table passports	permission denied for table passports

Последовательность действий и результат по каждой группе запросов:

1. Запрос 1: чтение employee_curation_stats_table разрешено обоим пользователям. Для guinea_pig это единственная разрешённая операция.
2. Запросы 2–4: запись в employee_curation_stats_table (INSERT, UPDATE, DELETE) запрещена обоим пользователям. Данные в таблице после запросов не изменились.

3. Запросы 5–8: работа с исходными таблицами. Для `guinea_pig` все операции запрещены, а `ex_curator` успешно выполняет чтение, вставку, изменение и удаление.
4. Запросы 9–10: обращения к посторонней таблице `passports` запрещены обоим пользователям как на чтение, так и на изменение.

Заключение

Приложение